

LAPORAN PRAKTIKUM
POSTTEST 7
ALGORITMA PEMROGRAMAN LANJUT



Disusun oleh:
Hamam Syamil (2509106073)
Kelas B2'25

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2025

1. Flowchart

1. Struktur Data & Inisialisasi

Program menggunakan Struct (User, Penyewa, Kost) untuk mengelompokkan data secara teratur dalam Array of Struct. Data penyewa dikelola sebagai nested struct di dalam data kost untuk keterhubungan informasi.

2. Menu Awal (Autentikasi)

Program menyediakan pilihan Register dan Login. Fungsi registerUser bertugas menambah akun dengan validasi kapasitas, sementara loginUser memverifikasi akun dengan batas tiga kali percobaan. Jika login gagal total, program berhenti; jika sukses, pengguna diarahkan ke menu utama.

3. Menu Utama (Operasi CRUD)

Menu ini dikelola dalam prosedur menuUtama menggunakan perulangan. Program menyediakan fitur:

- Menggunakan fungsi tambahData untuk input
- Prosedur lihatData untuk menampilkan tabel.
- Fungsi updateData yang memanfaatkan fungsi rekursif cariKamar untuk menemukan ID yang tepat sebelum data diubah.
- Fungsi hapusData memanfaatkan fungsi rekursif cariKamar untuk menemukan ID yang tepat sebelum data dihapus

4. Fitur Tambahan & Ringkasan

Terdapat fitur Ringkasan yang menggunakan fungsi rekursif totalHarga untuk menghitung pendapatan secara otomatis. Selain itu, program menerapkan Function Overloading pada prosedur tampilInfo, sehingga satu nama fungsi bisa menampilkan detail kamar dalam berbagai format sesuai kebutuhan.

5. Implementasi Pointer & Reference

- Pass by Reference (&)

Digunakan pada parameter fungsi pengelola data (seperti tambahData, hapusData, dan registerUser) agar perubahan jumlah data langsung tersinkronisasi dengan variabel asli di fungsi main tanpa perlu melakukan penyalinan data.

- **Pointer pada Struct (*)**

Digunakan dalam prosedur tampilInfo untuk mengakses anggota *struct* melalui alamat memorinya. Hal ini menunjukkan kemampuan program dalam melakukan manipulasi data pada tingkat memori (low-level) untuk efisiensi eksekusi.

6. Algoritma Pengurutan (Sorting)

Program telah dilengkapi dengan menu pengurutan data untuk memudahkan manajemen informasi kost. Terdapat tiga algoritma berbeda yang diimplementasikan:

- **Bubble Sort**

Mengurutkan nama penyewa dari A ke Z dengan membandingkan elemen yang berdekatan secara berulang.

- **Selection Sort**

Mengurutkan harga kamar dari yang tertinggi ke terendah dengan mencari nilai maksimal di setiap iterasi dan menempatkannya di posisi yang tepat.

- **Insertion Sort**

Mengurutkan ID kamar secara menaik dengan cara menyisipkan setiap elemen ke posisi yang benar dalam bagian array yang sudah terurut.

7. Implementasi Algoritma Searching

- **Cari Angka Menggunakan Binary Search**

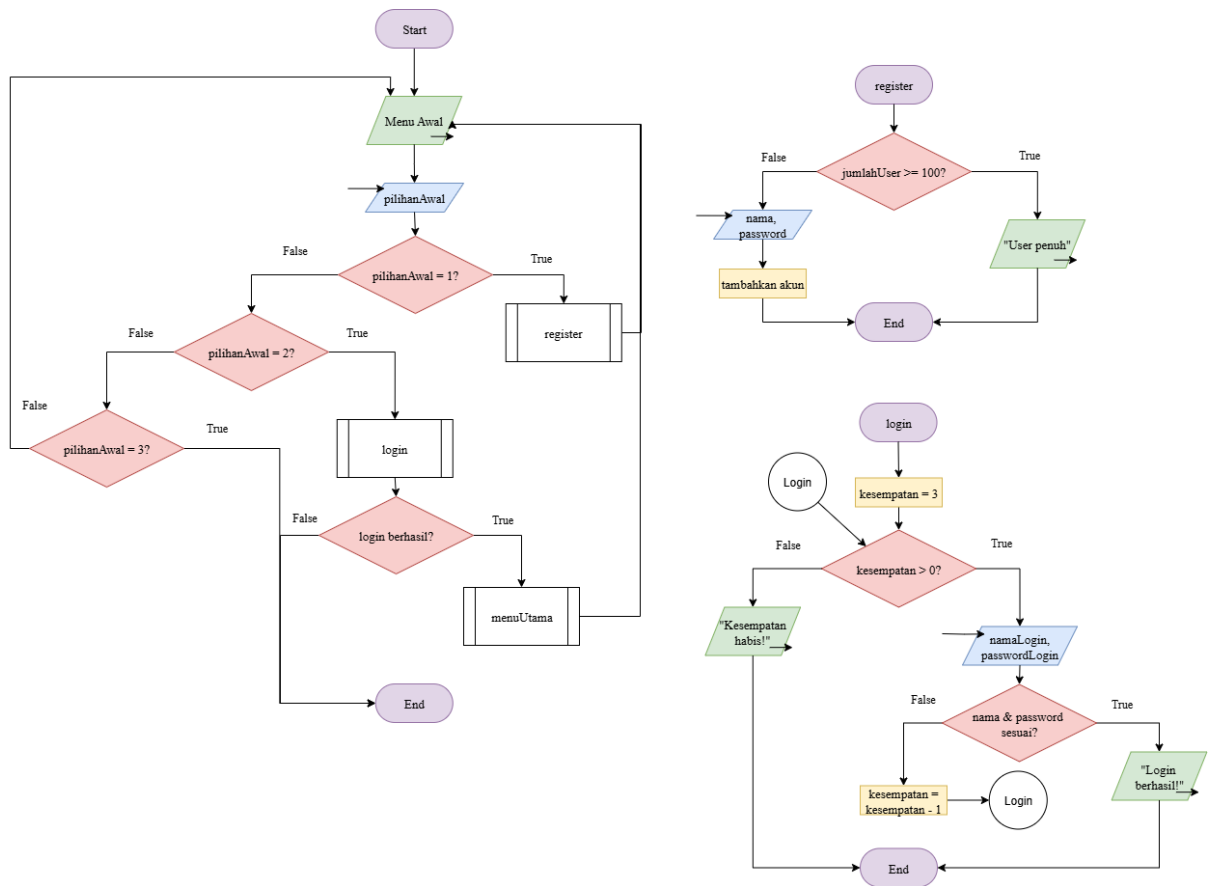
Digunakan untuk mencari ID Kamar. Algoritma ini bekerja pada data yang telah terurut (menggunakan insertionSortIDAsc terlebih dahulu) untuk membagi ruang pencarian menjadi dua secara terus-menerus.

- **Cari Kata Menggunakan Linear Search**

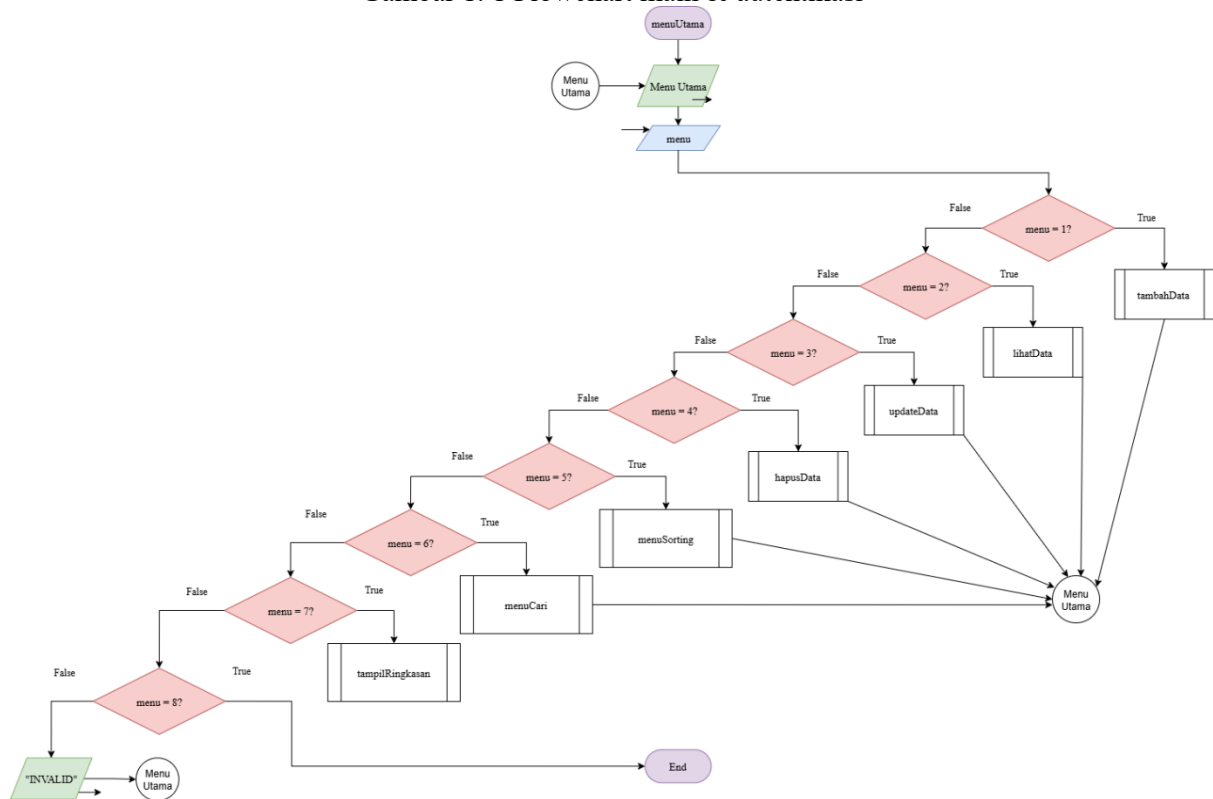
Digunakan untuk mencari Nama Penyewa. Algoritma ini memeriksa setiap elemen satu per satu dari awal hingga akhir, sangat efektif untuk pencarian string pada data yang tidak terurut.

8. Error Handling

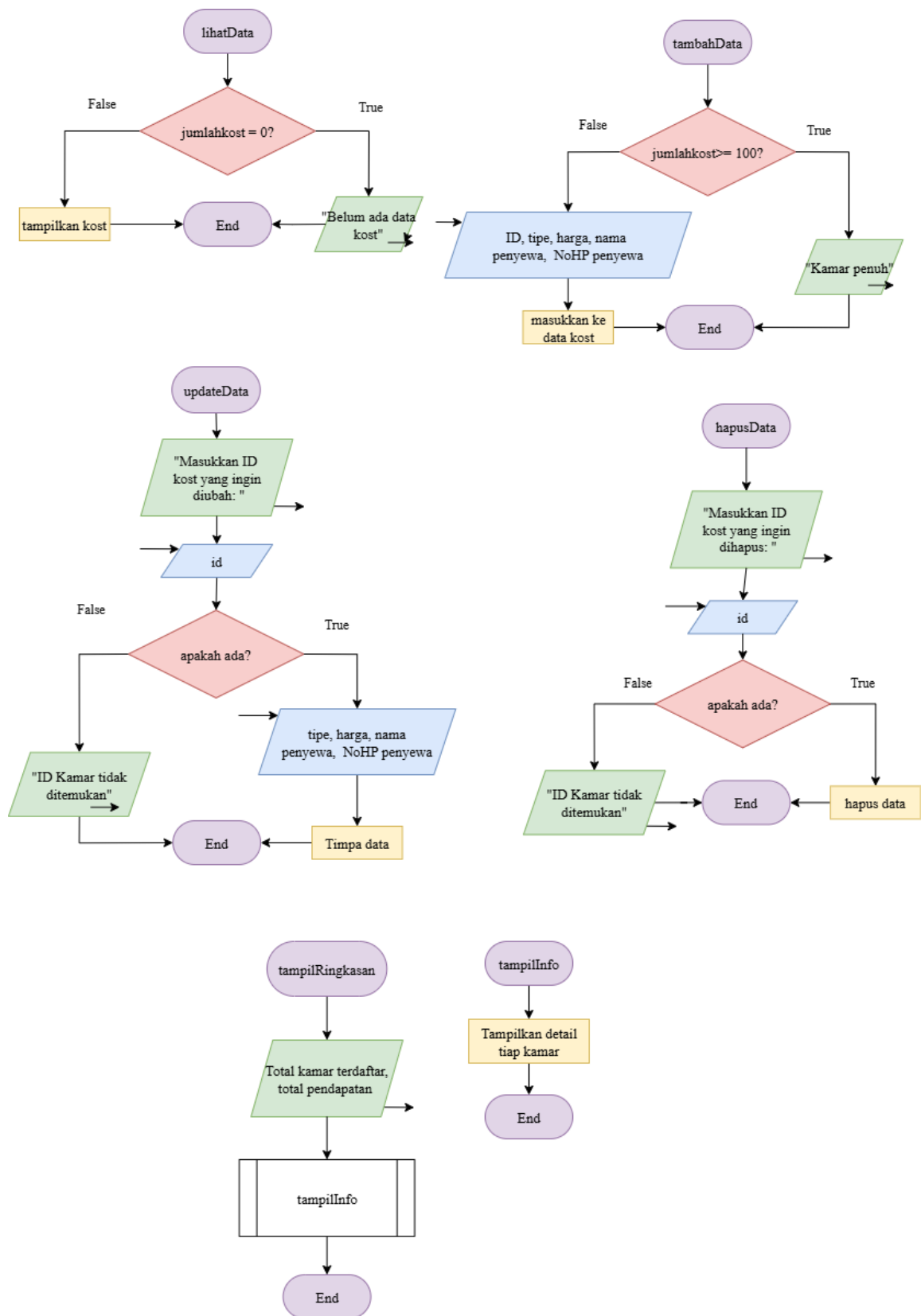
Program menerapkan Try, Throw, dan Catch. Ini digunakan untuk menangani kesalahan input (seperti input huruf pada kolom angka) atau pelanggaran aturan bisnis (seperti username kosong atau kapasitas penuh). Hal ini menjaga program agar tidak crash dan memberikan pesan error yang informatif kepada pengguna.



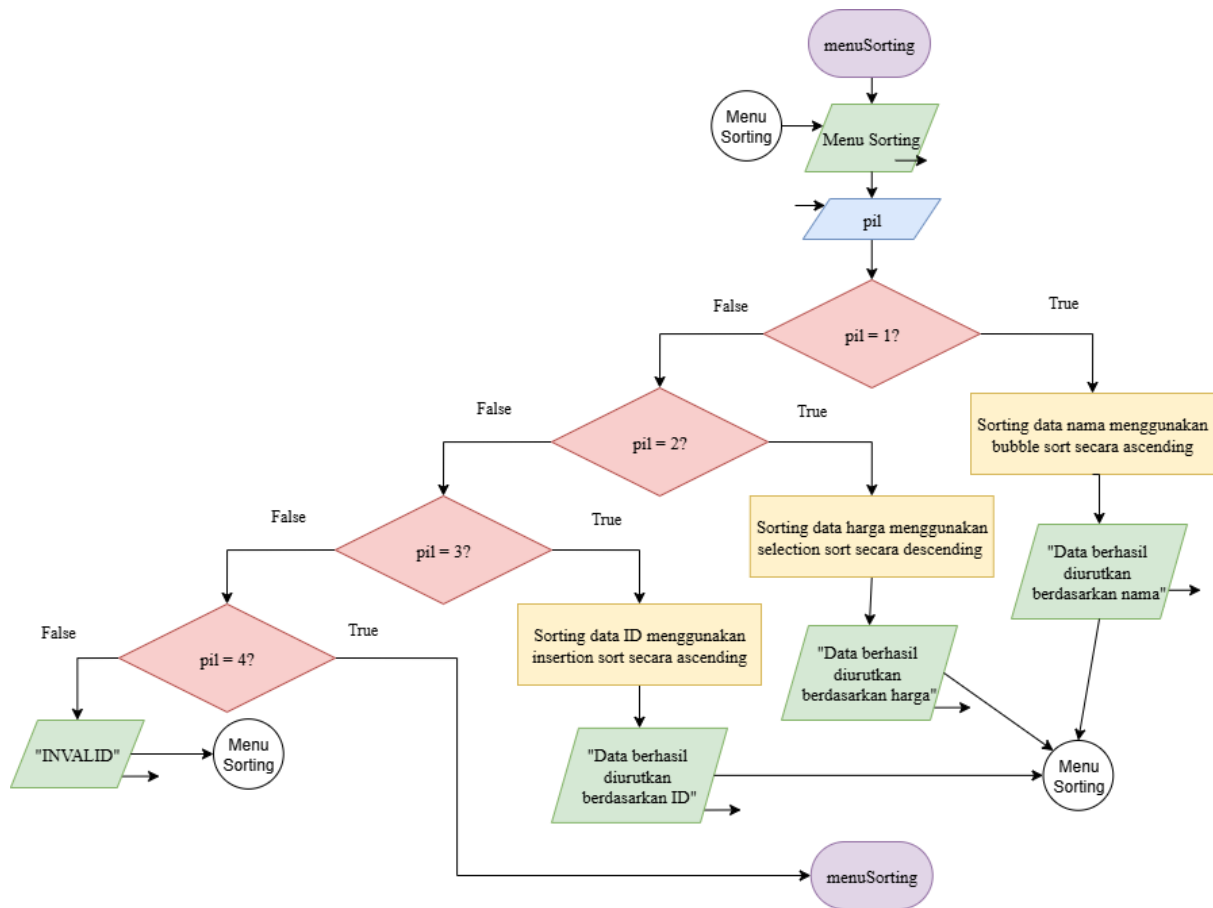
Gambar 1. 1 Flowchart main & autentikasi



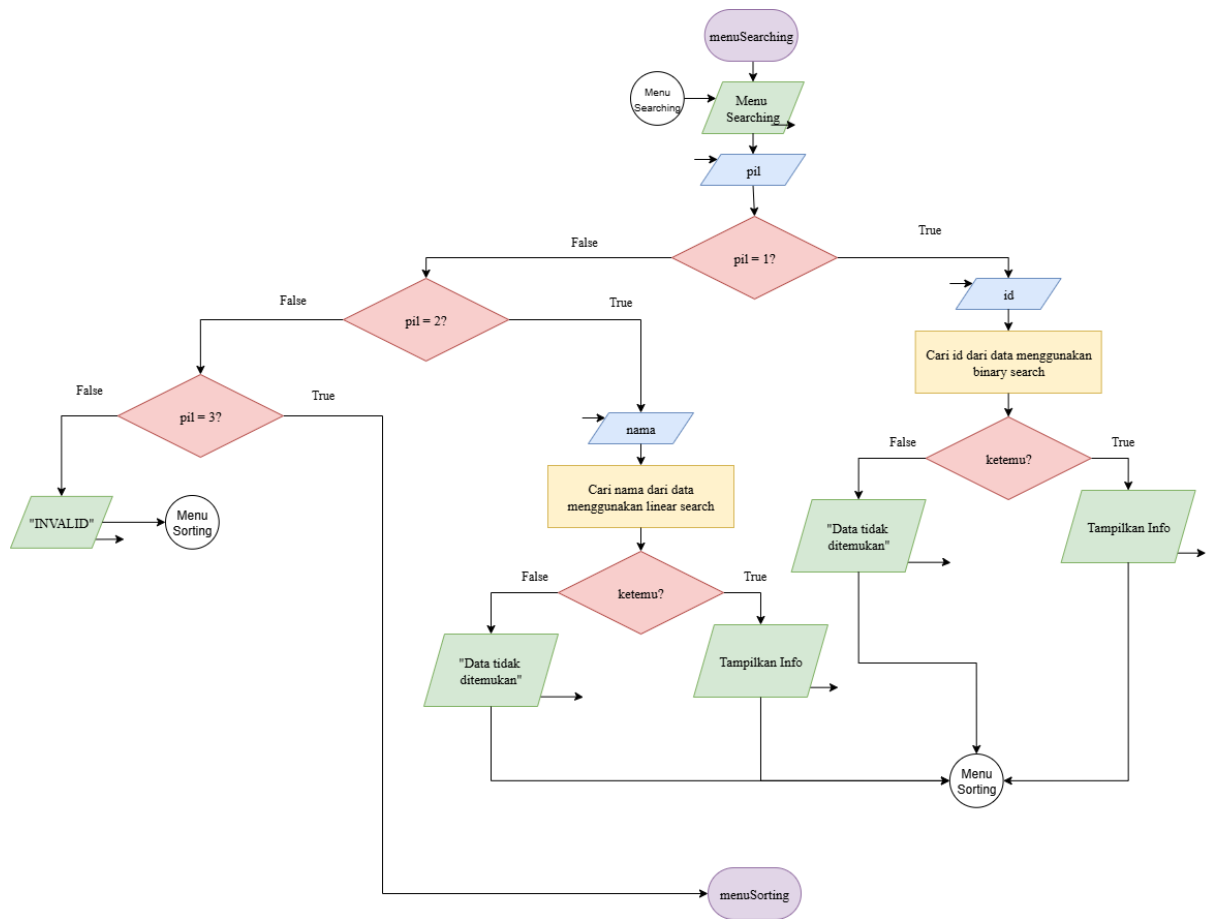
Gambar 1. 2 Flowchart Menu Utama



Gambar 1. 3 Flowchart CRUD



Gambar 1. 4 Flowchart Menu Sorting



Gambar 1. 5 Flowchart Menu Searching

2. Deskripsi Singkat Program

Program ini dirancang untuk mendigitalisasi pengelolaan data operasional kost agar lebih terstruktur, aman, dan efisien melalui sistem autentikasi Login dan Register yang terintegrasi dalam User Defined Library untuk modularitas kode yang lebih baik. Dengan implementasi fungsi CRUD (Create, Read, Update, Delete), pengelola dapat memanipulasi data penyewa secara praktis, yang didukung oleh Exception Handling menggunakan mekanisme try-throw-catch guna memvalidasi input serta mencegah terjadinya runtime error. Optimalisasi program diperkuat dengan penggunaan Pointer dan Pass by Reference untuk efisiensi manajemen memori, serta penerapan fungsi rekursif untuk penghitungan pendapatan total secara otomatis. Selain itu, integrasi berbagai algoritma Sorting (Bubble, Selection, Insertion) dan Searching (Linear dan Binary Search) memastikan pengorganisasian informasi serta penemuan data penyewa dapat dilakukan dengan kecepatan dan akurasi yang tinggi.

3. Source Code

- PENGATURAN STRUKTUR DATA & LIBRARY

Tempat mendefinisikan "wadah" data menggunakan struct yang dikemas dalam User Defined Library (kost_lib.h). Struktur User untuk akun, Penyewa untuk profil tamu, dan Kost sebagai data utama yang menggabungkan informasi secara nested (bersarang). Pemanfaatan library eksternal dan header file sendiri memastikan modularitas dan kemudahan pemeliharaan kode.

```
#ifndef KOST_LIB_H
#define KOST_LIB_H

#include <iostream>
#include <string>
#include <stdexcept>

using namespace std;

const int MAX_DATA = 100;

struct Penyewa {
    string nama;
    string noHP;
};

struct Kost {
    int idKamar;
    string tipe;
    int harga;
    Penyewa penyewa;
};

struct User {
    string nama;
    string password;
};

class KostException : public runtime_error {
public:
    KostException(const string& msg) : runtime_error(msg) {}
};

#endif
```


- FITUR ANTARMUKA & ESTETIKA

Berisi prosedur tampilHeader untuk menjaga konsistensi visual. Program menggunakan ANSI Color Codes (seperti \033[1;36m) untuk memberikan aksesoris warna pada judul dan status pesan, sehingga antarmuka terminal menjadi lebih menarik dan profesional.

```
void tampilHeader(string judul) {
    cout << "\n\033[1;36m" << string(10, '=') << " " << judul << " " << string(10, '=') << "\033[0m\n";
}
```

- FITUR AUTENTIKASI

Menggunakan logika Rekursif dan Searching tingkat lanjut. Fungsi totalHarga menjumlahkan seluruh harga kamar secara rekursif. Program menerapkan Binary Search untuk pencarian angka ID yang cepat (dengan syarat data terurut via insertionSortIDAsc) dan Linear Search untuk string nama, di mana keduanya menggunakan Pointer sebagai parameter untuk optimasi akses data pada alamat memori.

```
void registerUser(User users[], int &jumlahUser) {
    tampilHeader("REGISTER");
    try {
        if (jumlahUser >= MAX_DATA) throw KostException("Kapasitas user penuh!");

        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Username: ";
        getline(cin, users[jumlahUser].nama);
        if(users[jumlahUser].nama.empty()) throw KostException("Username tidak boleh kosong!");

        cout << "Password: ";
        getline(cin, users[jumlahUser].password);
        if(users[jumlahUser].password.length() < 4) throw KostException("Password minimal 4 karakter!");

        jumlahUser++;
        cout << "\033[1;32mRegistrasi berhasil!\033[0m\n";
    } catch (const KostException& e) {
        cout << "\033[1;31mError: " << e.what() << "\033[0m\n";
    }
}

bool loginUser(User users[], int jumlahUser) {
    tampilHeader("LOGIN");
    string namaLogin, passwordLogin;
    int kesempatan = 3;
    while (kesempatan > 0) {
        try {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Username: ";
            getline(cin, namaLogin);
            cout << "Password: ";
            getline(cin, passwordLogin);

            for (int i = 0; i < jumlahUser; i++) {
                if (users[i].nama == namaLogin && users[i].password == passwordLogin) {
                    cout << "\033[1;32mLogin berhasil! Selamat datang, " << namaLogin << "! \033[0m\n";
                    return true;
                }
            }
            kesempatan--;
            if (kesempatan > 0) {
                throw KostException("Username atau Password salah!");
            }
        } catch (const KostException& e) {
            cout << "\033[1;31m" << e.what() << " Sisa percobaan: " << kesempatan << "\033[0m\n\n";
        }
    }
    return false;
}
```

- FITUR PENCARIAN & PERHITUNGAN

Menggunakan logika Rekursif dan Searching tingkat lanjut. Fungsi totalHarga menjumlahkan seluruh harga kamar secara rekursif. Program menerapkan Binary Search untuk pencarian angka ID yang cepat (dengan syarat data terurut via insertionSortIDAsc) dan Linear Search untuk string nama, di mana keduanya menggunakan Pointer sebagai parameter untuk optimasi akses data pada alamat memori.

```
int totalHarga(Kost dataKost[], int index, int jumlahKost) {
    if (index >= jumlahKost) return 0;
    return dataKost[index].harga + totalHarga(dataKost, index + 1, jumlahKost);
}

int binarySearchID(Kost dataKost[], int n, int target) {
    insertionSortIDAsc(dataKost, n);
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (dataKost[mid].idKamar == target) return mid;
        if (dataKost[mid].idKamar < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int linearSearchNama(Kost dataKost[], int n, string target) {
    for (int i = 0; i < n; i++) {
        if (dataKost[i].penyewa.nama == target) {
            return i;
        }
    }
    return -1;
}
```

- FITUR UTAMA MANAJEMEN DATA

Mengelola siklus hidup data dengan pemanfaatan Pointer dan Reference untuk manipulasi data langsung pada memori asli:

- Alokasi data baru dengan validasi kapasitas array.
- Penyajian data terstruktur dalam format tabel menggunakan setw dari library <iomanip>.
- Modifikasi dan penghapusan elemen data spesifik berdasarkan hasil pencarian ID yang presisi, memastikan integritas data tetap terjaga selama proses pergeseran (shifting) array.

```

void updateData(Kost dataKost[], int jumlahKost) {
    tampilHeader("UPDATE DATA");
    try {
        int id;
        cout << "Masukkan ID Kamar: ";
        if(!(cin >> id)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            throw KostException("Input ID harus angka!");
        }

        int index = binarySearchID(dataKost, jumlahKost, id);
        if (index == -1) throw KostException("ID Kamar tidak ditemukan!");

        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Tipe Baru: "; getline(cin, dataKost[index].tipe);
        cout << "Harga Baru: "; cin >> dataKost[index].harga;
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Nama Penyewa Baru: "; getline(cin, dataKost[index].penyewa.nama);
        cout << "No HP Baru: "; getline(cin, dataKost[index].penyewa.noHP);
        cout << "\033[1;32mData berhasil diupdate!\033[0m\n";
    } catch (const KostException& e) {
        cout << "\033[1;31mError: " << e.what() << "\033[0m\n";
    }
}

void hapusData(Kost dataKost[], int &jumlahKost) {
    tampilHeader("HAPUS DATA");
    try {
        int id;
        cout << "Masukkan ID Kamar: ";
        cin >> id;

        int index = binarySearchID(dataKost, jumlahKost, id);
        if (index == -1) throw KostException("ID Kamar tidak ditemukan!");

        for (int j = index; j < jumlahKost - 1; j++) {
            dataKost[j] = dataKost[j + 1];
        }
        jumlahKost--;
        cout << "\033[1;32mData berhasil dihapus!\033[0m\n";
    } catch (const KostException& e) {
        cout << "\033[1;31mError: " << e.what() << "\033[0m\n";
    }
}

```

```

void tambahData(Kost dataKost[], int &jumlahKost) {
    tampilHeader("TAMBAH DATA");
    try {
        if (jumlahKost >= MAX_DATA) throw KostException("Kapasitas kamar penuh!");

        dataKost[jumlahKost].idKamar = jumlahKost + 1;
        cout << "ID: " << dataKost[jumlahKost].idKamar << endl;
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Tipe Kamar    : "; getline(cin, dataKost[jumlahKost].tipe);
        cout << "Harga          : ";
        if(!(cin >> dataKost[jumlahKost].harga)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            throw KostException("Input harga harus berupa angka!");
        }
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Nama Penyewa   : "; getline(cin, dataKost[jumlahKost].penyewa.nama);
        cout << "No HP Penyewa  : "; getline(cin, dataKost[jumlahKost].penyewa.noHP);

        jumlahKost++;
        cout << "\033[1;32mData berhasil ditambahkan!\033[0m\n";
    } catch (const KostException& e) {
        cout << "\033[1;31mError: " << e.what() << "\033[0m\n";
    }
}

void lihatData(Kost dataKost[], int jumlahKost) {
    tampilHeader("DATA KOST");
    if (jumlahKost == 0) {
        cout << "Belum ada data kost.\n";
        return;
    }
    cout << left << setw(5)
        << "ID" << setw(12)
        << "Tipe" << setw(12)
        << "Harga" << setw(20)
        << "Nama" << setw(15)
        << "No HP" << endl;
    cout << string(64, '-') << endl;
    for (int i = 0; i < jumlahKost; i++) {
        cout << left << setw(5)
            << dataKost[i].idKamar << setw(12)
            << dataKost[i].tipe << setw(12)
            << dataKost[i].harga << setw(20)
            << dataKost[i].penyewa.nama << setw(15)
            << dataKost[i].penyewa.noHP << endl;
    }
}

```

- **FITUR PENGURUTAN**

Implementasi tiga metode pengurutan berbeda untuk organisasi data yang lebih dinamis: Bubble Sort untuk nama (A-Z), Selection Sort untuk harga (Descending), dan Insertion Sort untuk ID (Ascending).

```
void bubbleSortNamaAsc(Kost dataKost[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (dataKost[j].penyewa.nama > dataKost[j + 1].penyewa.nama) {
                Kost temp = dataKost[j];
                dataKost[j] = dataKost[j + 1];
                dataKost[j + 1] = temp;
            }
        }
    }
}

void selectionSortHargaDesc(Kost dataKost[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int max_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (dataKost[j].harga > dataKost[max_idx].harga) {
                max_idx = j;
            }
        }
        Kost temp = dataKost[max_idx];
        dataKost[max_idx] = dataKost[i];
        dataKost[i] = temp;
    }
}

void insertionSortIDAsc(Kost dataKost[], int n) {
    for (int i = 1; i < n; i++) {
        Kost key = dataKost[i];
        int j = i - 1;
        while (j >= 0 && dataKost[j].idKamar > key.idKamar) {
            dataKost[j + 1] = dataKost[j];
            j = j - 1;
        }
        dataKost[j + 1] = key;
    }
}
```

- FITUR INFORMASI & RINGKASAN

Penerapan Function Overloading (melalui default parameters) dan Pointer pada Struct. Menggunakan fungsi tampilInfo yang mampu menampilkan kedalaman informasi berbeda secara fleksibel, serta akses data melalui alamat memori menggunakan operator panah (->) untuk efisiensi eksekusi.

```
void tampilInfo(Kost *k) {
    cout << "\033[1;33m[Info Kamar]\033[0m ID: "
          << k->idKamar << " | Tipe: "
          << k->tipe << " | Harga: Rp"
          << k->harga << " | Penyewa: "
          << k->penyewa.nama << endl;
}
```

- PROGRAM UTAMA

Pusat kendali alur program yang menggunakan perulangan do-while untuk menjaga stabilitas sesi pengguna. Fitur ini mengoordinasikan seluruh modul dan memastikan sumber daya memori dikelola dengan baik hingga pengguna memberikan instruksi keluar secara eksplisit.

```
int main() {
    User users[MAX_DATA];
    Kost dataKost[MAX_DATA];
    int jumlahUser = 0, jumlahKost = 0, pil;
    do {
        tampilHeader("SISTEM MANAGEMENT KOST");
        cout << "1. Register\n2. Login\n3. Keluar\nPilihan: ";
        if(!(cin >> pil)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }
        if (pil == 1) registerUser(users, jumlahUser);
        else if (pil == 2) {
            if (loginUser(users, jumlahUser)) menuUtama(dataKost, jumlahKost);
            else break;
        }
    } while (pil != 3);
    return 0;
}
```

4. Hasil Output

```
===== SISTEM MANAGEMENT KOST =====
1. Register
2. Login
3. Keluar
Pilihan: 1

===== REGISTER =====
Username: Admin_Kost
Password: 123
Error: Password minimal 4 karakter!

===== SISTEM MANAGEMENT KOST =====
1. Register
2. Login
3. Keluar
Pilihan: 1

===== REGISTER =====
Username: Admin_Kost
Password: 1234
Registrasi berhasil!

===== SISTEM MANAGEMENT KOST =====
1. Register
2. Login
3. Keluar
Pilihan: 2

===== LOGIN =====
Username: Admin_Kost
Password: 1234
Login berhasil! Selamat datang, Admin_Kost!
```

Gambar 4. 1 Output register & login

```
===== TAMBAH DATA =====
ID: 1
Tipe Kamar      : Eksklusif
Harga           : 2500000
Nama Penyewa    : Budi
No HP Penyewa   : 0812345
Data berhasil ditambahkan!
```

Gambar 4. 2 Output tambah data

```
===== DATA KOST =====
```

ID	Tipe	Harga	Nama	No HP
1	Eksklusif	2500000	Budi	0812345
2	Reguler	1500000	Caca	1500000
3	VIP	3000000	Andi	08333333
4	Besar	1750000	Alomani	08444444

Gambar 4. 3 Output lihat data

```
===== UPDATE DATA =====
Masukkan ID Kamar: 2
Tipe Baru: Reguler
Harga Baru: 1500000
Nama Penyewa Baru: Caca
No HP Baru: 08111111
Data berhasil diupdate!
```

Gambar 4. 4 Output update data

```
===== HAPUS DATA =====
Masukkan ID Kamar: 4
Data berhasil dihapus!
```

Gambar 4. 5 Output hapus data

```
===== MENU SORTING =====
1. Nama (Asc)
2. Harga (Desc)
3. ID (Asc)
```

Gambar 4. 6 Tampilan awal menu sorting

```
===== MENU SORTING =====
1. Nama (Asc)
2. Harga (Desc)
3. ID (Asc)
Pilihan: 1
Sorting selesai.
```

```
===== MENU UTAMA =====
1. Tambah 2. Lihat 3. Update 4. Hapus
5. Sorting 6. Cari 7. Ringkasan 8. Keluar
Pilihan: 2
```

```
===== DATA KOST =====
```

ID	Tipe	Harga	Nama	No HP
3	VIP	3000000	Andi	08333333
1	Eksklusif	2500000	Budi	0812345
2	Reguler	1500000	Caca	08111111

Gambar 4. 7 Output menu sorting 1 & lihat data kembali


```

===== MENU SORTING =====
1. Nama (Asc)
2. Harga (Desc)
3. ID (Asc)
Pilihan: 2
Sorting selesai.

```

```

===== MENU UTAMA =====
1. Tambah 2. Lihat 3. Update 4. Hapus
5. Sorting 6. Cari 7. Ringkasan 8. Keluar
Pilihan: 2

```

```

===== DATA KOST =====
ID   Tipe      Harga      Nama      No HP
-----
3    VIP        3000000    Andi      08333333
1    Eksklusif   2500000    Budi      0812345
2    Reguler     1500000    Caca      08111111

```

Gambar 4. 8 Output menu sorting 2 & lihat kembali

```

===== MENU SORTING =====
1. Nama (Asc)
2. Harga (Desc)
3. ID (Asc)
Pilihan: 3
Sorting selesai.

```

```

===== MENU UTAMA =====
1. Tambah 2. Lihat 3. Update 4. Hapus
5. Sorting 6. Cari 7. Ringkasan 8. Keluar
Pilihan: 2

```

```

===== DATA KOST =====
ID   Tipe      Harga      Nama      No HP
-----
1    Eksklusif   2500000    Budi      0812345
2    Reguler     1500000    Caca      08111111
3    VIP        3000000    Andi      08333333

```

Gambar 4. 9 Output menu sorting 3 & lihat kembali

```

===== MENU SEARCHING =====
1. Cari ID Kamar
2. Cari Nama Penyewa
Pilihan: 1
ID: 3
[Info Kamar] ID: 3 | Tipe: VIP | Harga: Rp3000000 | Penyewa: Andi

```

Gambar 4. 10 Output menu searching 1

```

===== MENU SEARCHING =====
1. Cari ID Kamar
2. Cari Nama Penyewa
Pilihan: 2
Nama: Budi
[Info Kamar] ID: 1 | Tipe: Eksklusif | Harga: Rp2500000 | Penyewa: Budi

```

Gambar 4. 11 Output menu searching 2

```

===== MENU UTAMA =====
1. Tambah 2. Lihat 3. Update 4. Hapus
5. Sorting 6. Cari 7. Ringkasan 8. Keluar
Pilihan: 7
Total Pendapatan: Rp7000000

```

Gambar 4. 12 Output ringkasan

```

===== SISTEM MANAGEMENT KOST =====
1. Register
2. Login
3. Keluar
Pilihan: 2

===== LOGIN =====
Username: salah
Password: ga benar
Username atau Password salah! Sisa percobaan: 2

Username: sala
Password: ga benar
Username atau Password salah! Sisa percobaan: 1

Username: salah
Password: ga benar

```

Gambar 4. 12 Output login gagal hingga 3x

5. Langkah-langkah GIT

5.1 GIT add

Perintah “git add” digunakan untuk menambahkan file ke staging area sebelum dilakukan commit. Pada gambar yang digunakan “git add .” agar semua perubahan di folder saat ini langsung ditambahkan sekaligus, tanpa harus memilih file satu per satu.

```
PS C:\Anomali\1_BELAJAR\APL\Praktikum\praktikum-apl\post-test\post-test-apl-7\2509106073_HAMMAMSYAMIL-PT-7> git add .
```

Gambar 5. 1 git add

5.2 GIT Commit

Perintah “git commit” digunakan untuk menyimpan perubahan dari staging area ke riwayat repository lokal. Pada gambar digunakan opsi -m, dipakai untuk memberi pesan singkat.

```
PS C:\Anomali\1_BELAJAR\APL\Praktikum\praktikum-apl\post-test\post-test-apl-7\2509106073_HAMMAMSYAMIL-PT-7> git commit -m "FINISH POSTTEST 7"
[main 4bc396c] FINISH POSTTEST 7
4 files changed, 351 insertions(+)
create mode 100644 post-test/post-test-apl-7/2509106073_HAMMAMSYAMIL-PT-7/kost_lib.h
create mode 100644 post-test/post-test-apl-7/2509106073_HAMMAMSYAMIL-PT-7/main.cpp
create mode 100644 post-test/post-test-apl-7/2509106073_HAMMAMSYAMIL-PT-7/main.exe
create mode 100644 post-test/post-test-apl-7/2509106073_HAMMAMSYAMIL-PT-7/tempCodeRunnerFile.cpp
```

Gambar 5. 2 git commit

5.3 GIT Push

Perintah “git push” digunakan untuk mengirim commit dari repository lokal ke repository remote.

```
PS C:\Anomali\1_BELAJAR\APL\Praktikum\praktikum-apl\post-test\post-test-apl-7\2509106073_HAMMAMSYAMIL-PT-7> git push
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 4 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 57.59 KiB | 3.03 MiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/Aeloxyl/praktikum-apl
6f31ba6..4bc396c main -> main
```

Gambar 5. 3 git push